

Software Integration & Maintenance

Fahim Foysal Abak

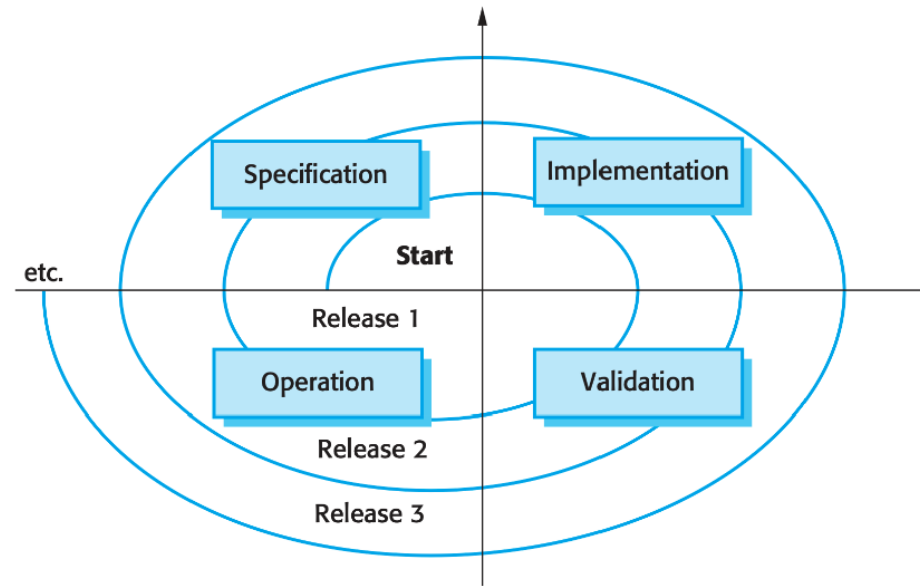
Lecture, Dept. of CSE

Dhaka International University

Software Maintenance

Agenda:

- Introduction to Software Maintenance
- Need for Maintenance
- Types of Maintenance
- Maintenance Process
- Maintenance Cost
- Maintenance Strategies
- Factors Affecting Maintenance Cost
- Legacy Systems



Software Maintenance

Software maintenance is the process of modifying a software system after delivery to **correct faults, improve performance, or adapt it to a changed environment.**

Example:

A university uses a **Result Management System** to calculate and publish student grades.

Problem:

After deployment, students report that **CGPA is calculated incorrectly** when a course is repeated.

Maintenance Action:

Developers identify a logic error in the grading algorithm and fix it.

Software Maintenance

Problem:

The university upgrades its database from **MySQL** to **PostgreSQL**.

Maintenance Action:

The software is modified so it can run correctly with the new database.

Problem:

Users request a **new feature** to download results as a PDF.

Maintenance Action:

Developers add a “Download PDF” option and improve the user interface.

Problem:

The system code is old, poorly structured, and hard to understand.

Maintenance Action:

The team **refactors the code**, improves documentation, and removes unused modules.

Why Software Maintenance?

- ❑ Business requirements change
- ❑ Bugs appear after deployment
- ❑ Environment & technology evolve
- ❑ Performance improvements needed

Software Evolution

Software evolution refers to the ongoing development of software systems after initial deployment.

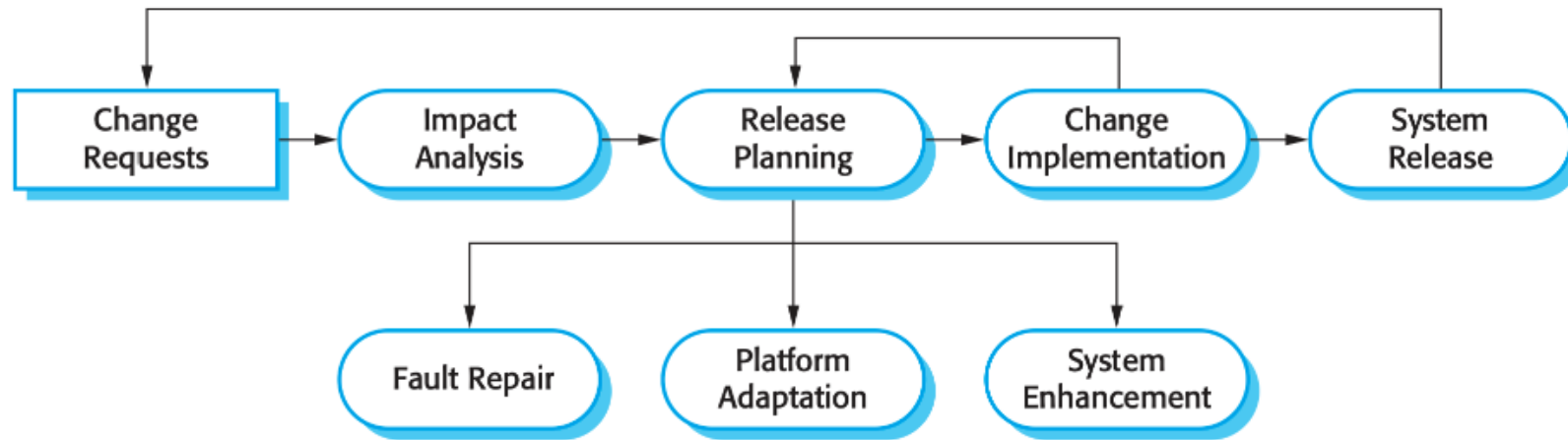


Figure: The software evolution process

Lehman's Laws of Software Evolution

- ☐ Continuing Change
- ☐ Increasing Complexity
- ☐ Large Program Evolution
- ☐ Organizational Stability
- ☐ Conservation of Familiarity
- ☐ Continuing growth
- ☐ Declining quality
- ☐ Feedback system

Lehman's Laws of Software Evolution

Law	Description
Continuing change	A program that is used in a real-world environment must necessarily change, or else become progressively less useful in that environment.
Increasing complexity	As an evolving program changes, its structure tends to become more complex. Extra resources must be devoted to preserving and simplifying the structure.
Large program evolution	Program evolution is a self-regulating process. System attributes such as size, time between releases, and the number of reported errors is approximately invariant for each system release.
Organizational stability	Over a program's lifetime, its rate of development is approximately constant and independent of the resources devoted to system development.

Lehman's Laws of Software Evolution

Law	Description
Conservation of familiarity	Over the lifetime of a system, the incremental change in each release is approximately constant.
Continuing growth	The functionality offered by systems has to continually increase to maintain user satisfaction.
Declining quality	The quality of systems will decline unless they are modified to reflect changes in their operational environment.
Feedback system	Evolution processes incorporate multiagent, multiloop feedback systems and you have to treat them as feedback systems to achieve significant product improvement.

Types of Software Maintenance

Fault repairs: Coding errors are usually relatively cheap to correct; design errors are more expensive as they may involve rewriting several program components. Requirements errors are the most expensive to repair because of the extensive system redesign which may be necessary.

Environmental adaptation: This type of maintenance is required when some aspect of the system's environment such as the hardware, the platform operating system, or other support software changes. The application system must be modified to adapt it to cope with these environmental changes.

Functionality addition: This type of maintenance is necessary when the system requirements change in response to organizational or business change. The scale of the changes required to the software is often much greater than for the other types of maintenance.

Types of Software Maintenance

- ❑ **Corrective:** Fixes errors and bugs discovered after deployment.
- ❑ **Adaptive:** Changes made to keep software usable in a changing environment.
- ❑ **Perfective:** Enhancements to improve performance or maintainability.
- ❑ **Preventive:** Changes to prevent future problems.

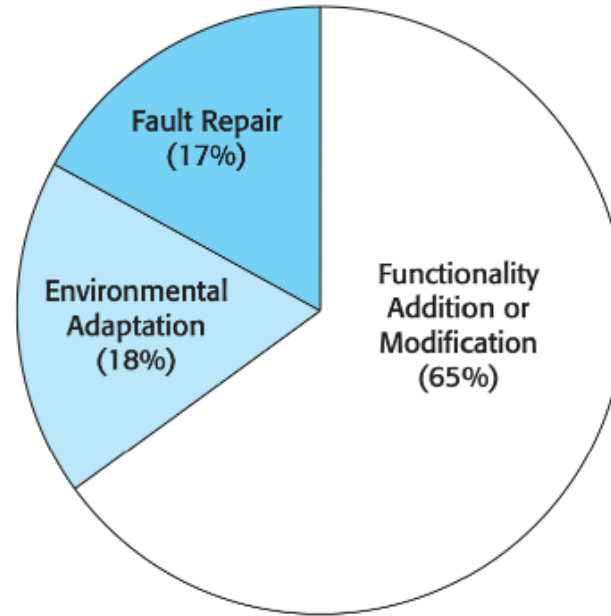


Figure: Maintenance effort distribution

Maintenance Cost

Software maintenance takes up a higher proportion of IT budgets than new development (roughly two-thirds maintenance, one-third development).

The relative costs of **maintenance** and **new development** vary from one application domain to another. Guimaraes (1983) found that the maintenance costs for business application systems are broadly comparable with system development costs.

In embedded real-time systems, maintenance costs can be up to four times higher than development costs.

These systems require very high reliability and performance, which leads to:

- ❑ Tightly coupled modules
- ❑ Strong interdependencies between components

Because of tight coupling, making changes becomes difficult and expensive.

Factors Increasing Software Maintenance Cost

It is usually more expensive to add functionality after a system is in operation than it is to implement the same functionality during development. The reasons for this are:

Team Stability

- ☐ After delivery, the original development team often disperses.
- ☐ New maintenance staff lack system and design knowledge.
- ☐ Extra time is needed to understand the system before making changes.

Poor Development Practice

- ☐ Maintenance is usually handled under a separate contract.
- ☐ Often done by a different company than the developer.
- ☐ Developers may not focus on maintainability to save development effort.

Staff Skills

- ☐ Maintenance is often assigned to junior or less experienced staff.
- ☐ Maintenance work has a poor reputation among engineers.
- ☐ Older systems may use obsolete languages unfamiliar to current staff.

Factors Increasing Software Maintenance Cost

Program Age and Structure

- ❑ As software ages, repeated changes degrade program structure.
- ❑ Poor structure makes the system harder to understand and modify.
- ❑ Older systems may lack modern software engineering practices.
- ❑ Many were optimized for efficiency rather than readability.
- ❑ Documentation may be missing, outdated, or inconsistent.
- ❑ Weak or absent configuration management makes it difficult to find correct versions of components.

Software Maintenance Strategies

- ☐ Quick Fix: Fast changes to correct issues but may increase complexity.
- ☐ Iterative enhancement: Regular planned updates improving quality and functionality.
- ☐ Component replacement
- ☐ System re-engineering: Improving structure without changing functionality.

Software Reengineering

Software Reengineering focuses on improving the internal structure of existing software without changing its external behavior.

According to Sommerville:

- ❑ Software reengineering is the examination and alteration of a system
- ❑ It is followed by reconstruction in a new form
- ❑ The system's functionality remains the same
- ❑ Main goal: reduce future maintenance costs

Why Software Reengineering is Needed

Reengineering is required because:

- ☐ Legacy systems are business-critical
- ☐ Systems are difficult to maintain
- ☐ Code structure has degraded over time
- ☐ Documentation is outdated or missing
- ☐ Replacing systems is risky and expensive

Reengineering vs Replacement

Reengineering	Replacement
Lower risk	High risk
Incremental	Big bang
Preserves knowledge	Knowledge lost
Faster results	Long development time

Sommerville recommends **reengineering** when system quality is poor but **functionality is valuable**.

Software Reengineering Activities

Four main reengineering activities:

- 1.Source Code Translation
- 2.Reverse Engineering
- 3.Program Structure Improvement
- 4.Data Reengineering

Legacy systems

- ❑ **Legacy systems** are **old systems** that are still useful and are sometimes **critical** to business operation.
- ❑ They may be implemented using **outdated languages** and **technology** or may use other systems that are **expensive to maintain**.
- ❑ Often their structure has been **degraded** by change and **documentation** is missing or out of date.

Legacy system management

Scrap the system completely: This option should be chosen when the system is not making an effective contribution to business processes. This commonly occurs when business processes have changed since the system was installed and are no longer reliant on the legacy system.

Leave the system unchanged and continue with regular maintenance: This option should be chosen when the system is still required but is fairly stable and the system users make relatively few change requests.

Legacy system management

Reengineer the system to improve its maintainability: This option should be chosen when the system quality has been degraded by change and where a new change to the system is still being proposed. This process may include developing new interface components so that the original system can work with other, newer systems.

Replace all or part of the system with a new system This option should be chosen when factors, such as new hardware, mean that the old system cannot continue in operation or where off-the-shelf systems would allow the new system to be developed at a reasonable cost.

Legacy system management

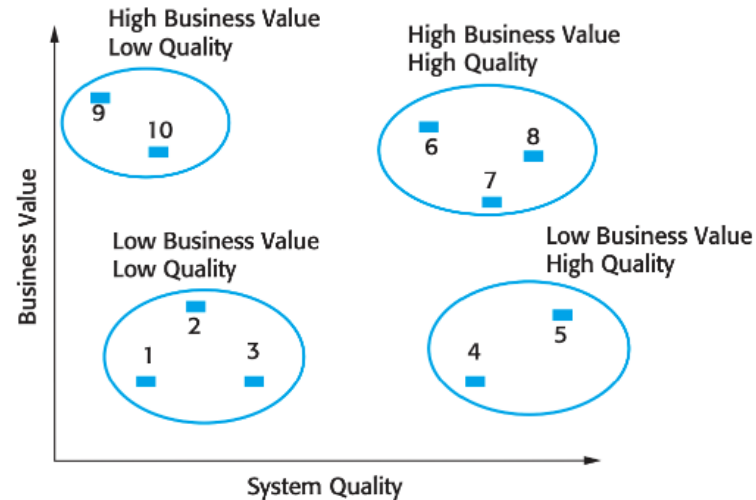


Figure: An example of a legacy system assessment

- 1. Low quality, low business value** Keeping these systems in operation will be expensive and the rate of the return to the business will be fairly small. These systems should be scrapped.
- 2. Low quality, high business value** These systems are making an important business contribution so they cannot be scrapped. However, their low-quality means that it is expensive to maintain them. These systems should be reengineered to improve their quality. They may be replaced, if a suitable off-the-shelf system is available.

Legacy system management

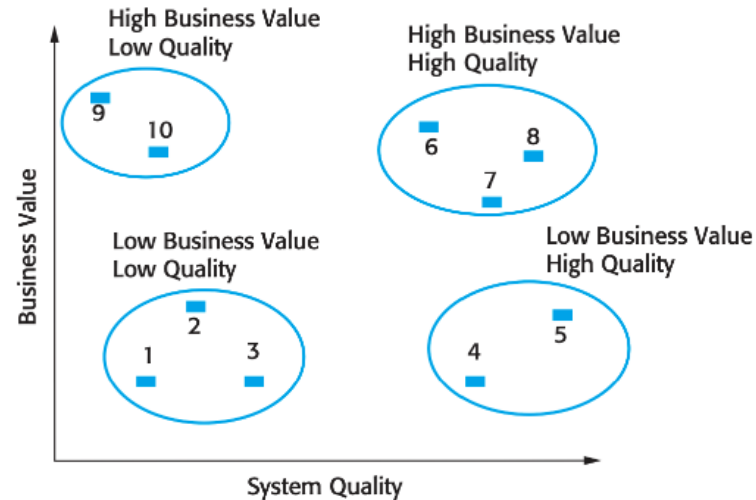


Figure: An example of a legacy system assessment

3. High quality, low business value These are systems that don't contribute much to the business but which may not be very expensive to maintain. It is not worth replacing these systems so normal system maintenance may be continued if expensive changes are not required and the system hardware remains in use. If expensive changes become necessary, the software should be scrapped.

Legacy system management

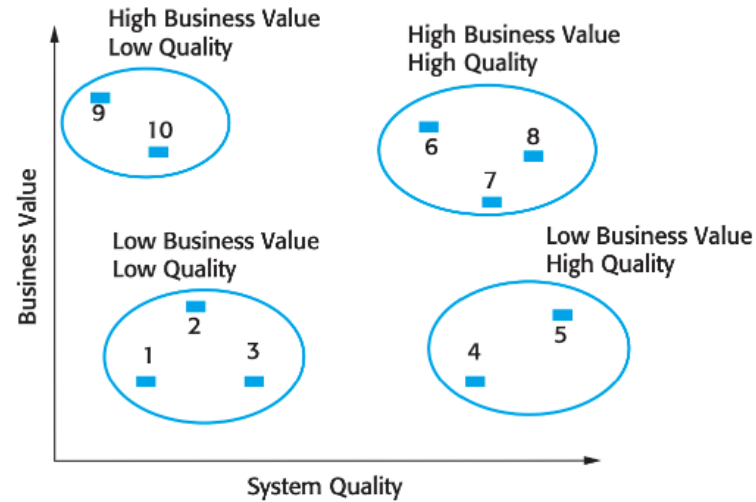


Figure: An example of a legacy system assessment

4. **High quality, high business value** These systems have to be kept in operation. However, their high-quality means that you don't have to invest in transformation or system replacement. Normal system maintenance should be continued.

Thank You.